

Kerberos/NFS

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook을 단일 기준으로 삼는다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 소유 경계
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	keytab 발급, ticket 발급, RPCSEC_GSS, NAS 권한 판정, 설정 근거
운영	계정/컨테이너 생성, timer 확인, mount-KVNO 장애를 처리할 때	명령, 점검표, 모니터링 코드, 수동 repair/rotation

한눈에 보는 구조

관리 서버(uidctl)

- AD DC: 사용자·RFC2307·keytab 생성
- 계산 호스트: root-only keytab + ccache refresh timer
- NAS/storage: NFS service keytab + 사용자 home
- container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd

-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구현의 중요한 경계는 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.
- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc`와 읽기 전용 `/etc/krb5.conf`만 bind mount한다.

- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer` 를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share` 다.
- `addr=100.100.100.120` 은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...` 로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5` 다. `krb5i` 와 `krb5p` 는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 관측한 home owner UID
= container UID
```

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown` 만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈 소유 경계

작업	소유 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	<code>user-lifecycle</code>
공통 host Kerberos/NFS desired state와 fstab	<code>server-state</code>
NAS NFS service account·SPN·service keytab의 수동 변경	<code>kerberos-nfs</code>
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	<code>monitoring</code>
ccache 소비, 시작 전 credential 확인, restricted sudo	<code>container-images</code>

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

단일 기준 문서와 코드

- [FARM canonical runbook](#)

- [LAB canonical runbook](#)
- [user-lifecycle Kerberos 구현](#)
- [container 생성 구현](#)
- [host Kerberos/NFS desired state](#)
- [Kerberos/NFS keytab health check](#)
- [NFS GSS monitoring](#)

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.
- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

flowchart LR

```
M["관리 서버<br/>uidctl + Ansible"]
AD["Samba AD DC / KDC<br/>사용자·그룹·SPN<br/>RFC2307·KVNO"]
H["FARM/LAB 계산 호스트<br/>user keytab<br/>refresh timer<br/>rpc.gssd + kernel NFS"]
C["사용자 container<br/>process + ccache<br/>KRB5CCNAME"]
NAS["NAS / storage<br/>svcgssd + NFS server<br/>service keytab<br/>winbind/idmap"]
MON["monitoring<br/>readiness·KVNO·mount<br/>canary·forensics"]

M -->|"samba-tool / exportkeytab"| AD
M -->|"root-only keytab 설치"| H
M -->|"home과 numeric owner 준비"| NAS
AD <-->|"AS/TGS, directory lookup"| H
H -->|"ccache bind mount"| C
C -->|"파일 I/O"| H
H <-->|"NFSv4 + RPCSEC_GSS"| NAS
NAS -->|"RFC2307 UID/GID 조회"| AD
MON -.->|"읽기 전용 상태 수집"| AD
MON -.-> H
MON -.-> NAS
```

구성요소별 책임

위치	객체/프로세스	하는 일
관리 서버	uidctl, Ansible runner	계정 생성 transaction을 조정하고 keytab을 필요한 host로 전달
AD DC	사용자·그룹 객체, krbtgt, KDC, samba-tool	principal과 RFC2307 속성 저장, keytab export, TGT/service ticket 발급
계산 호스트	사용자 keytab	ticket을 새로 발급할 수 있는 장기 자격증명; root만 읽음
계산 호스트	decs-krb-refresh@.service/.timer	keytab으로 ccache를 만들고 갱신하여 원자적으로 교체
계산 호스트	kernel NFS client, rpc.gssd	호출 UID의 credential로 NFS RPCSEC_GSS context 생성
컨테이너	사용자 process, bind-mounted ccache	KRB5CCNAME 이 가리키는 ticket을 사용하고 NFS 파일 I/O 수행
NAS/storage	svcgssd, NFS server, service keytab	nfs/<fqdn>@<realm> service ticket을 수락하고 NFS 요청 처리
NAS/storage	Samba/winbind/idmap	Kerberos 이름을 AD RFC2307 UID/GID로 해석하여 파일 권한 판정

3. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.

```
sequenceDiagram
    autonumber
    participant CLI as 관리 서버 uidctl
    participant AD as Samba AD DC
    participant Host as target 계산 호스트
    participant NAS as NAS/storage
    participant Docker as Docker container

    CLI->>AD: 사용자·그룹 생성/조회
    CLI->>AD: uidNumber, gidNumber, group membership 설정
    CLI->>AD: domain exportkeytab --principal=user@REALM
    AD-->>CLI: 임시 keytab
    CLI->>Host: /etc/decs-krb/keytabs/user.keytab<br/>root:root 0400 설치
    CLI->>NAS: home 생성 및 numeric UID:GID 준비
    CLI->>Host: refresh env/service/timer 설치·활성화
    Host->>AD: kinit -k -t user.keytab
    AD-->>Host: 사용자 TGT가 든 ccache
    Host->>Host: /run/user/uid/krb5cc로 원자 교체
    CLI->>Host: 실제 NFS 쓰기 검증
    CLI->>Docker: ccache dir와 krb5.conf만 bind mount
```

실제 생성 구현은 다음 순서로 동작한다.

1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 `uidNumber`, `gidNumber` 를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 `0400` 으로 만든다.
4. target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
5. NAS/storage home을 동일 UID/GID로 준비한다.
6. target host에 ccache directory, refresh helper, service/timer를 만든다.
7. host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.
8. 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity](#)와 [keytab 생성](#), [container 생성 transaction](#), [credential 경로 모델](#)에서 확인한다.

4. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.

```
sequenceDiagram
    autonumber
    participant P as container 사용자 process
    participant K as host kernel NFS client
    participant G as host rpc.gssd
    participant KDC as AD KDC
    participant S as NAS svcgssd/NFS
    participant ID as AD RFC2307 / winbind

    P->>K: open/read/write (호출 UID 포함)
    K->>G: RPCSEC_GSS credential upcall
    G->>G: /run/user/uid/krb5cc에서 사용자 TGT 확인
    G->>KDC: nfs/nas.farm.decs.internal service ticket 요청
    KDC-->>G: NFS service ticket
    G-->>K: GSS security context
    K->>S: NFSv4 RPC + Kerberos credential
    S->>S: service keytab으로 ticket 복호화
    S->>ID: principal을 RFC2307 UID/GID로 해석
    ID-->>S: numeric UID/GID와 group
    S-->>K: 파일 권한 판정 결과
    K-->>P: I/O 결과
```

인증 실패 위치에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/idmap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

5. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능

구분	keytab	ccache
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	/run/user/<uid>/krb5cc, 사용자 0600
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 `klist` 검증을 끝낸 뒤 소유권 `uid:gid`, mode `0600`을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

6. Ticket 갱신 설계

`decs-krb-refresh@<username>.timer`는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

유효 ccache 있음

- └─ renew deadline이 margin보다 멀 -> 임시 복사본에서 `kinit -R`
- └─ deadline 임박/renew 실패 -> root-only keytab으로 새 ticket 발급

발급 결과 -> `klist` 검증 -> `uid:gid 0600` -> 최종 ccache로 atomic move

기본 재발급 여유는 `DECS_KRB_REISSUE_BEFORE_SECONDS=86400` (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

7. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account `NAS$`가 아니라 전용 AD service account `svc-nfs-farm`이 소유한다.

```
nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
nfs/NAS@FARM.DECS.INTERNAL
```

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

8. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share
service SPN:  nfs/nas.farm.decs.internal@FARM.DECES.INTERNAL
transport:    addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

9. 보안 flavor와 권한 모델

- `sec=krb5`: 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i`: 인증 + RPC payload 무결성.
- `sec=krb5p`: 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5`다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted`를 함께 사용한다. 다만 restricted sudo는 완전한 sandbox가 아니므로 강한 격리가 필요하면 sudo와 `SYS_ADMIN`, host bind mount 범위도 별도로 줄여야 한다.

10. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook

관심사	기준 코드/문서
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기와 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 구현된 Docker/FARM·LAB 흐름의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 해당 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 `Kerberos command builder`에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
  /etc/decs-krb/keytabs/<username>.keytab \
  /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
  klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하는 것이 현재 구현이다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |
  grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='

docker exec --user <username> <container> \
  sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,
_netdev,exec,nouser 0 0
```

핵심은 source가 FQDN이라는 점이다.

```
올바름: nas.farm.decs.internal:/volume1/share
잘못됨: 100.100.100.120:/volume1/share
잘못됨: 192.168.2.30:/volume1/share
정상: mount option 또는 findmnt 결과의 addr=100.100.100.120
```

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal과 이름이 맞지 않는다. `192.168.2.30` 은 NAS 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
findmnt -T /home/tako8/share -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n '/\/home\/tako8\/share/,+4p'
getent hosts nas.farm.decs.internal
```

desired fstab 변경은 [server-state playbook](#)이 소유한다. 이미 mount된 legacy superblock은 유지보수 창에 clean unmount/remount한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')
sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "${short}@FARM.DECS.INTERNAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. NAS service account와 KVNO 운영

현재 정책

- NFS SPN 소유자: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```

cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'

```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재

- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 FARM config에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair`는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd`를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

8. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery

무엇을 확인하는가	대표 상태/지표	코드
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

9. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME`, ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

10. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt`, `klist`, checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

11. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.